

ASSIGNMENT - 5 REPORT

Introduction

This assignment investigates the principles of parallel and distributed computing using the Message Passing Interface (MPI). The objective is to understand how computational problems behave under parallel execution and to evaluate scalability, communication overhead, synchronization cost, and load balancing strategies.

Theory

Parallel Speedup

Speedup is defined as: $S(p) = T_1 / T_p$

Where: T_1 = serial execution time, T_p = parallel execution time with p processes. Ideal speedup is linear: $S(p) = p$. However, in real systems, speedup is limited by communication and synchronization overhead.

Efficiency

$E(p) = S(p) / p$

Efficiency measures how effectively processes are utilized.

Amdahl's Law

Amdahl's Law states that the maximum theoretical speedup is limited by the serial portion of a program: $S(p) = 1 / (f + (1-f)/p)$ where f = fraction of serial work. This explains why speedup eventually saturates as processes increase.

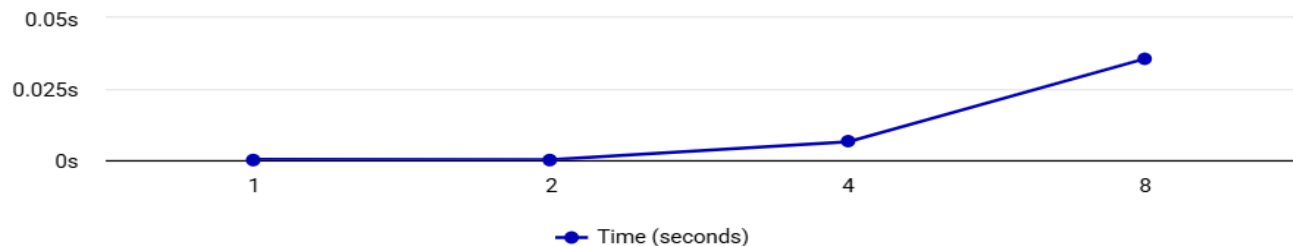
Q1 – DAXPY (Data Parallelism)

Description

Implements the operation: $X = a \cdot X + Y$

The data is divided among processes using: MPI_Scatter, Local computation, MPI_Gather

Version	Time (seconds)
Serial	0.000213
NP 1	0.000224
NP 2	0.000139
NP 4	0.006450
NP 8	0.035673



Observations

Best performance at NP=2. Performance degrades significantly at NP=4 and NP=8.

Inference

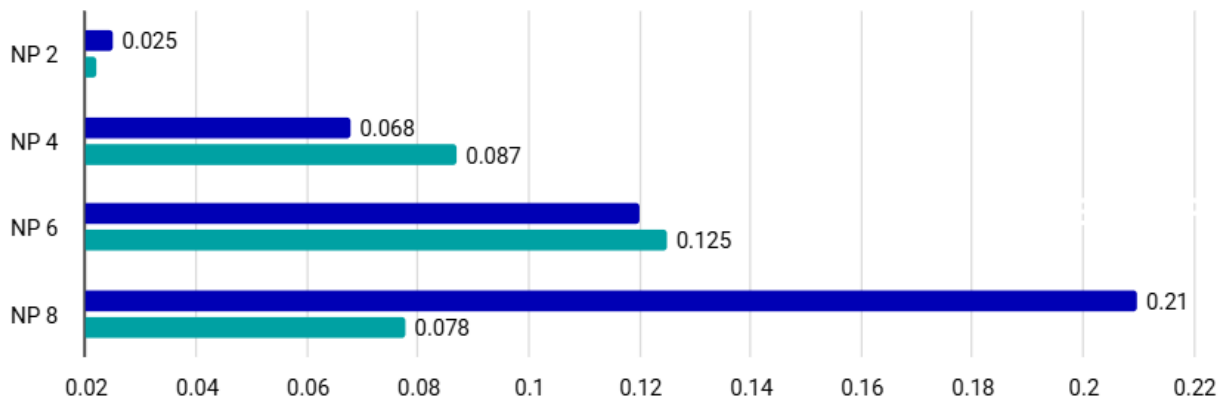
For small data sizes, communication overhead (Scatter/Gather) dominates computation, causing poor scalability at higher process counts.

Q2 – Broadcast Race (MyBcast vs MPI_Bcast)

Description

Two broadcast methods were compared: Manual broadcast using MPI_Send and MPI_Recv, Optimized MPI_Bcast

NP	Manual (s)	MPI_Bcast (s)
2	0.025098	0.022356
4	0.068553	0.087492
6	0.120855	0.125833
8	0.211277	0.078700



Observations

Manual broadcast time increases steadily as NP increases. MPI_Bcast is significantly faster at NP=8.

Inference

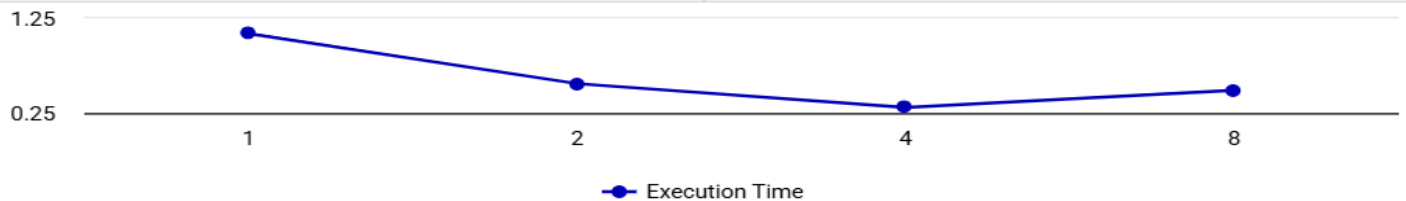
MPI_Bcast scales better than manual broadcast because it uses optimized tree-based communication instead of linear sends.

Q3 – Distributed Dot Product

Description

Total dot product of a large dataset (500 million operations) was computed using: MPI_Bcast, Local computation, MPI_Reduce

Version	Time (s)
Serial	0.969270
NP 1	1.098865
NP 2	0.565433
NP 4	0.312094
NP 8	0.492262



Observations

Strong speedup from NP=1 to NP=4. Performance decreases at NP=8.

Inference

High computation-to-communication ratio gives good scalability initially, but synchronization and reduction overhead limit performance at higher processes.

Q4 – Prime Number Search (Master–Slave)

Description

Dynamic scheduling was implemented: Master assigns numbers, Slaves test primality, Results returned dynamically

Version	Time (s)
Serial	0.019546
NP 2	0.285959
NP 4	0.194860
NP 8	0.194034

Observations

Parallel execution is slower than serial. Slight improvement from NP=2 to NP=4/8.

Inference

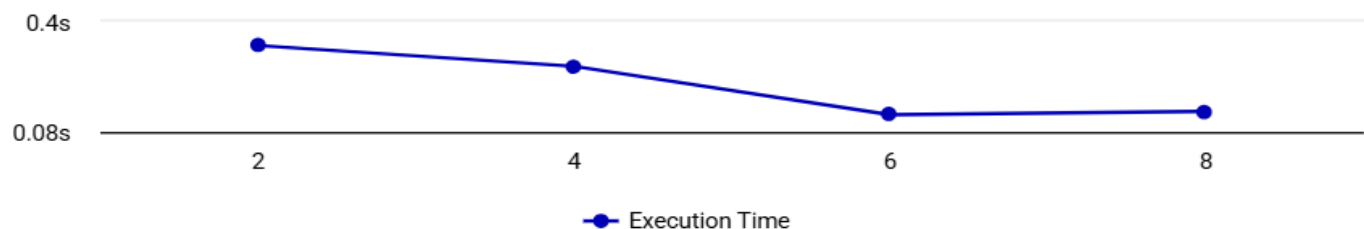
Frequent communication between master and slaves introduces overhead, making parallel execution inefficient for this problem size.

Q5 – Perfect Number Search (Master–Slave)

Description

Similar to Q4 but checks for perfect numbers by computing divisor sums.

Version	Time (s)
Serial	0.113418
NP 2	0.333947
NP 4	0.279995
NP 6	0.139397
NP 8	0.141374



Observations

Parallel initially slower than serial. Performance improves significantly at NP=6 and NP=8.

Inference

As workload increases, parallel computation begins to offset communication overhead, improving scalability.

Conclusion

This assignment demonstrates that parallel performance is highly dependent on the balance between computation and communication. While computation-heavy tasks such as distributed dot product exhibit strong scalability, communication-intensive tasks suffer from overhead that limits speedup. The broadcast comparison confirms that optimized collective operations outperform manual message passing. Additionally, dynamic master–slave models improve load balancing but introduce coordination bottlenecks. Overall, the results validate Amdahl's Law, showing that increasing process count does not guarantee proportional speedup due to inherent serial components and communication overhead.